Wood Block Shape Sorting

Wood Block Shape Sorting

1. Course Content
2. Preparation
3. Case Demonstration
4. Color Block Color Calibration
5. Core Source Code Analysis

1. Course Content

This feature implements the program acquiring images through the camera, recognizing the shapes of the wooden blocks in the image and grasping wooden blocks of the target shape, and placing them in a set position.

> **Important**
>
> The grasping point calculation methods differ between the depth camera and the USB monocular camera: the depth camera obtains the spatial coordinates of the target center point through the depth map, while the monocular camera grasps objects at a fixed z-axis height through parameters

2. Preparation

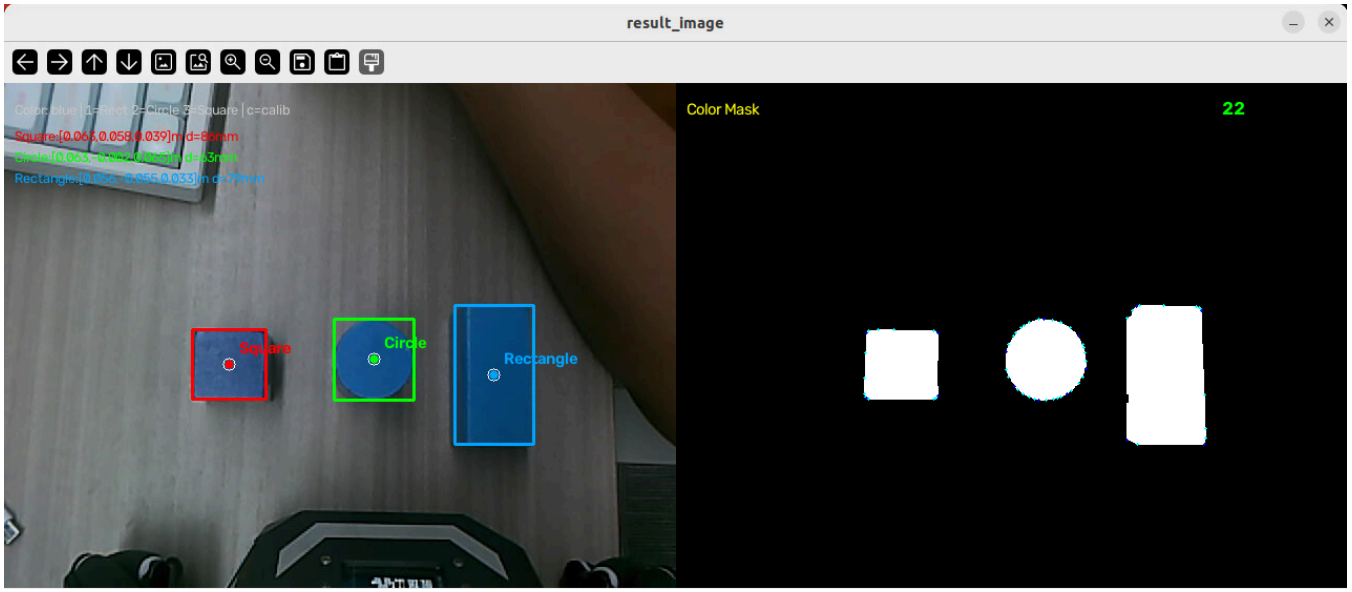
The wooden block used in this section: **30x30x30mm, 30x30x60mm**

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=shape_sort
```

- Place the three shapes of wooden blocks within the camera's field of view

3. Case Demonstration

- Press R, G, B, or Y to select the color to detect
- Press 1 to sort the rectangle, press 2 to sort the circle, and press 3 to sort the square. After sorting, the robotic arm will return to the observation pose, ready for the next sorting round



Note: If it is a USB monocular camera, **the spatial position cannot be calculated through the camera**. If a cylinder needs to be grasped, adjust `usb_object_height` in the parameter file to 0.06m; the factory default of 0.03m is the height of the square and rectangle on the tabletop.

4. Color Block Color Calibration

Refer to the color calibration steps in the previous section's tutorial 【Color Block Color Sorting】; the calibration method is the same.

5. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/shape_sort.py
```

_detect_shapes_by_color function (shape detection):

In monocular camera mode, this function detects the shape type of the target-color region (square, rectangle, circle) based on the LAB color mask.

Color mask generation:

- `cv2.cvtColor(rgb_image, cv2.COLOR_RGB2LAB)`: converts the RGB image to the LAB color space; LAB is more robust to lighting changes
- `color_lab_map[self._target_color]`: obtains the LAB threshold range of the target color `[l_min, a_min, b_min, l_max, a_max, b_max]`
- `_create_lab_mask(...)`: generates a binary mask according to the LAB thresholds; the target-color region is white and the rest is black

Morphological processing:

- `cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))`: creates an elliptical structuring element

for the closing operation

- `cv2.morphologyEx(mask, cv2.MORPH_CLOSE, ...)`: the closing operation fills small holes inside the mask and connects adjacent regions
- `cv2.morphologyEx(mask, cv2.MORPH_OPEN, ...)`: the opening operation removes small noise points and smooths the contour boundaries

Contour extraction and shape classification:

- `cv2.findContours(mask, cv2.RETR_EXTERNAL, ...)`: extracts the list of outer contours of the mask
- `cv2.contourArea(cnt)`: calculates the contour area, filtering out noise contours smaller than `min_block_area`
- `cv2.convexHull(cnt)`: calculates the convex hull of the contour, eliminating concave parts
- `cv2.approxPolyDP(hull, 0.035 * peri, True)`: polygon approximation, where `0.035 * peri` is the approximation accuracy (3.5% of the perimeter), returning the vertices of the approximated polygon
- `corner_num = len(approx)`: the number of vertices, used to determine the shape type
- `cv2.minAreaRect(hull)`: calculates the minimum enclosing rectangle, obtaining the center coordinates and size

Shape determination rules:

- **4 vertices**: calculates the adjacent side lengths; if the difference between adjacent sides is less than 50 pixels it is a **Square**, otherwise it is a **Rectangle**
- **5 or more vertices**: calculates the fill ratio `fill_ratio = area / (rect_w * rect_h)`; the theoretical fill ratio of a circle is π/4 ≈ 0.785, and if it is less than 0.82 it is determined to be a **Circle**
- **Other cases**: determined to be **None** (unknown shape)

Result visualization:

- `cv2.drawContours(mask_vis, hull, -1, (255, 255, 0), 2)`: draws the cyan convex hull contour on the mask visualization image
- `cv2.drawContours(mask_vis, approx, -1, (255, 0, 0), 2)`: draws the blue approximated polygon contour
- Returns the `blocks` list (containing shape, center coordinates, area, rectangle, contour, number of vertices) and the `mask_vis` visualization image

```
    def _detect_shapes_by_color(self, rgb_image):
        """Monocular mode: detect the shape of the target-color region based on the LAB
color mask
        Classification rules: 4 vertices with equal adjacent sides -> Square; 4 vertices
with unequal sides -> Rectangle;
        many vertices with fill ratio < 0.82 -> Circle
        Returns:
            blocks: list of dict, mask_vis: mask visualization image
        """
        blocks = []
        if self._target_color is None or self._target_color not in self.color_lab_map:
            mask_vis = np.zeros(rgb_image.shape, dtype=np.uint8)
            return blocks, mask_vis
```

```
        lab = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2LAB)
        l_min, a_min, b_min, l_max, a_max, b_max = \
            self.color_lab_map[self._target_color]
        mask = self._create_lab_mask(
            lab, l_min, a_min, b_min, l_max, a_max, b_max
        )

        kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_close, iterations=3)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_open, iterations=2)

        mask_vis = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)

        contours, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
        )

        for cnt in contours:
            area = cv2.contourArea(cnt)
            if area < self.min_block_area:
                continue

            hull = cv2.convexHull(cnt)
            peri = cv2.arcLength(hull, True)
            approx = cv2.approxPolyDP(hull, 0.035 * peri, True)
            corner_num = len(approx)
            rect = cv2.minAreaRect(hull)

            if corner_num == 4:
                side_lengths = np.array([
                    np.linalg.norm(approx[i][0] - approx[(i + 1) % 4][0])
                    for i in range(4)
                ])
                # Adjacent-side difference < 50px is treated as a square
                if np.allclose(side_lengths[1], side_lengths[0], atol=50):
                    shape_type = "Square"
                else:
                    shape_type = "Rectangle"
            elif corner_num >= 5:
                # Circle fill-ratio check: circle area / bounding rectangle area ≈ π/4 ≈
0.785
                rect_w, rect_h = rect[1]
                if rect_w > 0 and rect_h > 0:
                    fill_ratio = area / (rect_w * rect_h)
                    shape_type = "Circle" if fill_ratio < 0.82 else "None"
                else:
                    shape_type = "Circle"
            else:
                shape_type = "None"

            center = rect[0]
```

```
            cx, cy = int(center[0]), int(center[1])

            cv2.drawContours(mask_vis, hull, -1, (255, 255, 0), 2)
            cv2.drawContours(mask_vis, approx, -1, (255, 0, 0), 2)

            blocks.append({
                'shape': shape_type,
                'center': (cx, cy),
                'area': area,
                'rect': rect,
                'contour': hull,
                'n_vertices': corner_num,
            })

        return blocks, mask_vis
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

Parameter Description:

- `shape_grasp_distance: [0.1, 0.0]`: the expected grasping distance during chassis alignment `[x, y]` (meters); x is the front-back direction distance, and y is the left-right direction offset
- `grasp_offset: [0.0, 0.0, 0.0, 85.0]`: grasping offset `[x, y, z, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position; pitch set to 85° means the gripper grasps almost vertically downward
- `gripping_angle: 140`: the angle (degrees) when the gripper is closed; the larger the value, the tighter the gripper
- `observation_pose: [91, 95, 35, -34, 90, 30]`: the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `min_block_area: 2000`: minimum area of the color mask contour (pixels), filtering out noise detection results with too small an area
- `usb_object_height: 0.03`: the fixed height (meters) of the object in the measure coordinate frame in USB monocular mode; the default 0.03m corresponds to the height of the square and rectangle on the tabletop; it needs to be adjusted to 0.06m when grasping a cylinder
- `usb_camera_measure: [0.012, 0.0, 0.013]`: the pose compensation value `[x, y, z]` (meters) in USB monocular mode, correcting the deviation between the camera mounting position and the origin of the measure coordinate frame

```yaml
shape_sort:
  ros__parameters:
    shape_grasp_distance: [0.1, 0.0]     # x,y axis alignment distance (meters)
    grasp_offset: [0.0,0.0,0.0,85.0]     # x,y,z axis offset distance and included angle, compensating for servo backlash, link rigidity degradation, and special grasping requirements
    gripping_angle: 140                    # gripper grasping angle, in degrees, integer
    observation_pose: [91, 95, 35, -34, 90, 30]
    min_block_area: 2000                  # minimum color mask contour area (pixels), filtering small noise
    usb_object_height: 0.03                # fixed height of the object in the measure frame in usb monocular mode (meters)
    usb_camera_measure: [0.012, 0.0, 0.013] # pose compensation in usb monocular mode (meters)
```